

Practical 12

1. Title

Implementation of Global Exception Handling in Student REST API using Spring Boot

2. Aim

To implement global exception handling in a Spring Boot REST API by creating a custom exception and handling it using `@ControllerAdvice` and `@ExceptionHandler`.

3. Learning Objectives

- To understand REST API development using Spring Boot
- To learn custom exception creation
- To implement global exception handling
- To understand the use of `@ControllerAdvice`
- To return proper HTTP status codes

4. Software Requirements

- Framework: Spring Boot
- Language: Java
- IDE: Eclipse / IntelliJ
- Tool: Postman
- Build Tool: Maven
- JDK: 8+

5. Theory Background

- REST APIs are used to communicate between client and server
- Exceptions may occur when data is not found
- **Custom Exception** helps handle specific errors
- `@ControllerAdvice` provides global exception handling
- `@ExceptionHandler` handles specific exceptions
- HTTP status codes improve API response clarity

6. Assignment Questions

11.1: Exception Handling in Student REST API Enhance an existing Spring Boot Student REST API by implementing global exception handling. Create a custom exception named **StudentNotFoundException** that is thrown when a student record with a given ID is not found. Use **@ControllerAdvice** and **@ExceptionHandler** to handle the exception globally and return an appropriate HTTP status code with an error message. Test the API using Postman for valid and invalid student IDs.

Input:

Student.java

```
package com.shriyash.mca.model;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String course;

    public Student() {
    }

    public Student(String name, String course) {
        this.name = name;
        this.course = course;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }
}
```

```
public void setName(String name) {  
    this.name = name;  
}  
  
public String getCourse() {  
    return course;  
}  
  
public void setCourse(String course) {  
    this.course = course;  
}  
}
```

StudentRepository.java

```
package com.shriyash.mca.repository;
```

```
import org.springframework.data.jpa.repository.JpaRepository;  
import org.springframework.stereotype.Repository;
```

```
import com.shriyash.mca.model.Student;
```

```
@Repository
```

```
public interface StudentRepository extends JpaRepository<Student, Long> {  
}
```

StudentNotFoundException.java

```
package com.shriyash.mca.exception;
```

```
public class StudentNotFoundException extends RuntimeException {
```

```
    public StudentNotFoundException(Long id) {  
        super("Student record with ID " + id + " not found.");  
    }  
}
```

studentControoler.java

```
package com.shriyash.mca.controller;
```

```
import java.util.List;
```

```
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.http.HttpStatus;  
import org.springframework.http.ResponseEntity;  
import org.springframework.web.bind.annotation.GetMapping;
```

Python Programming Lab – MCA Sem II

```
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.shriyash.mca.exception.StudentNotFoundException;
import com.shriyash.mca.model.Student;
import com.shriyash.mca.repository.StudentRepository;

@RestController
@RequestMapping("/api/students")
public class StudentController {

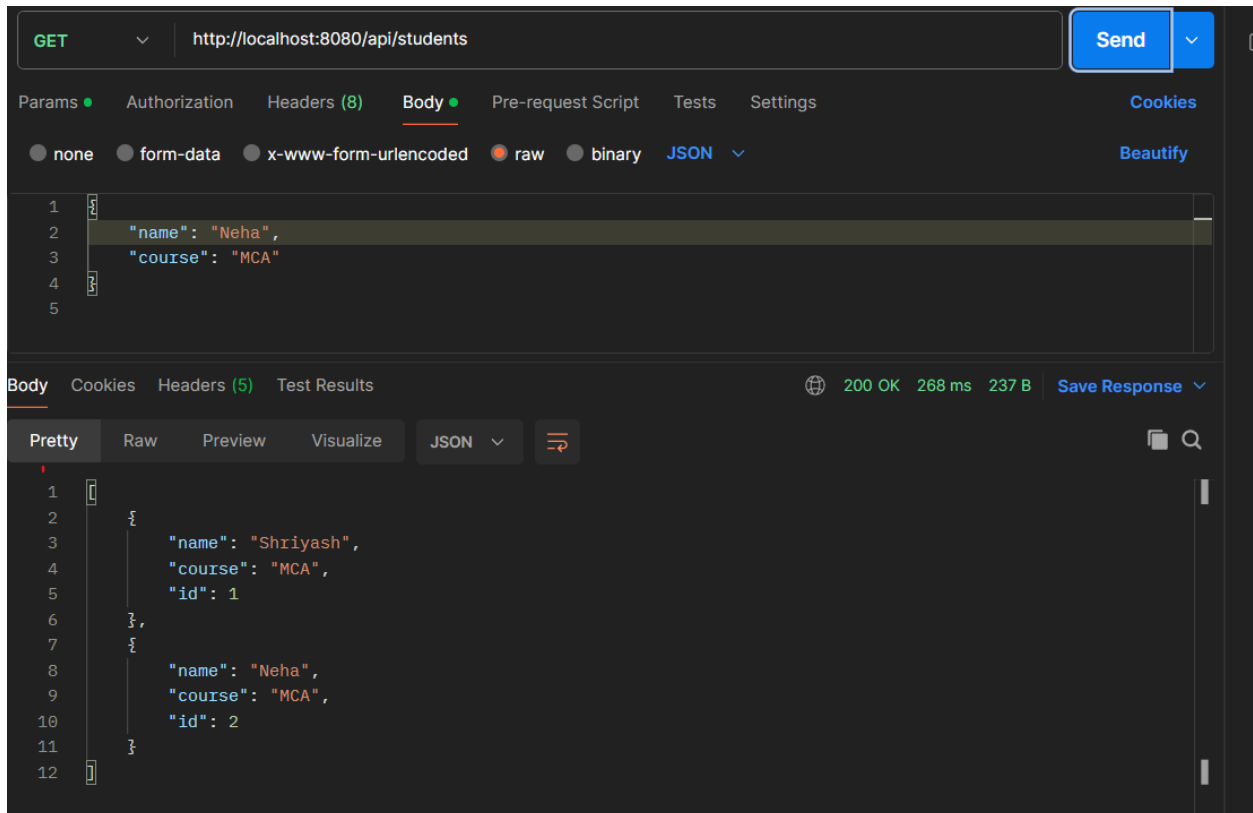
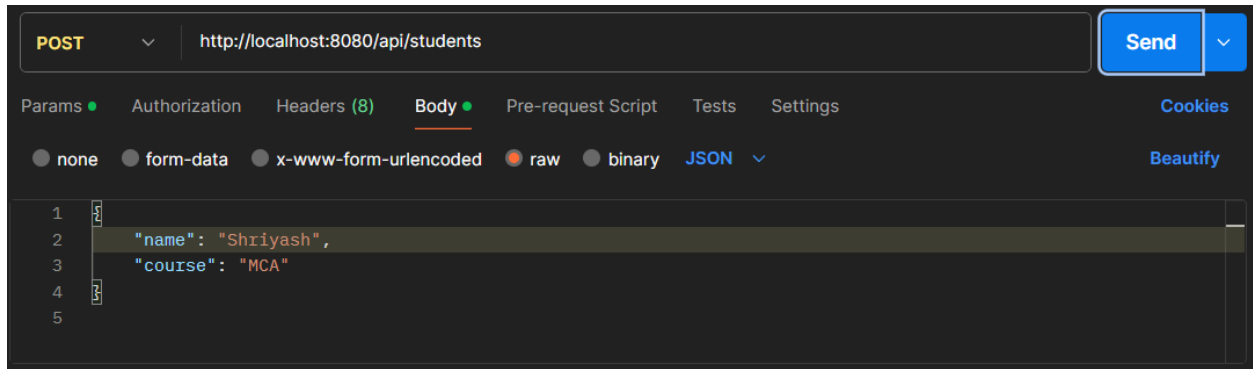
    @Autowired
    private StudentRepository studentRepository;

    @PostMapping
    public ResponseEntity<Student> createStudent(@RequestBody Student student) {
        Student savedStudent = studentRepository.save(student);
        return new ResponseEntity<>(savedStudent, HttpStatus.CREATED);
    }

    @GetMapping
    public List<Student> getAllStudents() {
        return studentRepository.findAll();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Student> getStudentById(@PathVariable Long id) {
        Student student = studentRepository.findById(id)
            .orElseThrow(() -> new StudentNotFoundException(id));
        return ResponseEntity.ok(student);
    }
}
```

Output:



11.2: Transaction Management in Bank Account API Develop a simple REST API using Spring Boot to simulate money transfer between two bank accounts. Create methods to debit one account

and credit another account within a service method annotated with @Transactional. If an error occurs during the transfer process, ensure that all operations are rolled back. Test the transaction behavior by simulating a failure during the transfer and verify that account balances remain unchanged.

Input:

AccountService.java

```
package com.shriyash.mca.service;
```

```
import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.stereotype.Service;
```

```
import org.springframework.transaction.annotation.Transactional;
```

```
import com.shriyash.mca.model.Account;
```

```
import com.shriyash.mca.repository.AccountRepository;
```

```
@Service
```

```
public class AccountService {
```

```
    @Autowired
```

```
    private AccountRepository accountRepository;
```

```
    public Account createAccount(Account account) {
```

```
        return accountRepository.save(account);
```

```
    }
```

```
    public Account getAccount(Long id) {
```

```
        return accountRepository.findById(id).orElseThrow(() -> new RuntimeException("Account not found"));
```

```
    }
```

```
    @Transactional
```

```
    public void transferMoney(Long fromAccountId, Long toAccountId, Double amount, boolean simulateError) {
```

```
        Account fromAccount = accountRepository.findById(fromAccountId)
```

```
            .orElseThrow(() -> new RuntimeException("Source account not found"));
```

```
        Account toAccount = accountRepository.findById(toAccountId)
```

```
            .orElseThrow(() -> new RuntimeException("Destination account not found"));
```

```
        if (fromAccount.getBalance() < amount) {
```

```
            throw new RuntimeException("Insufficient balance");
```

```
        }
```

```
fromAccount.setBalance(fromAccount.getBalance() - amount);
accountRepository.save(fromAccount);

if (simulateError) {
    throw new RuntimeException("Simulated error during money transfer! Transaction should
rollback.");
}

toAccount.setBalance(toAccount.getBalance() + amount);
accountRepository.save(toAccount);
}
}
```

AccountRepository.java

```
package com.shriyash.mca.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import com.shriyash.mca.model.Account;

@Repository
public interface AccountRepository extends JpaRepository<Account, Long> {
}
```

Account.java

```
package com.shriyash.mca.model;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Account {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String accountHolderName;
    private Double balance;

    public Account() {
    }
}
```

```
public Account(String accountHolderName, Double balance) {
    this.accountHolderName = accountHolderName;
    this.balance = balance;
}

public Long getId() {
    return id;
}

public void setId(Long id) {
    this.id = id;
}

public String getAccountHolderName() {
    return accountHolderName;
}

public void setAccountHolderName(String accountHolderName) {
    this.accountHolderName = accountHolderName;
}

public Double getBalance() {
    return balance;
}

public void setBalance(Double balance) {
    this.balance = balance;
}
}
```

GlobalExceptionHandler.java

```
package com.shriyash.mca.exception;
```

```
import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Map;
```

```
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ExceptionHandler;
```

```
@ControllerAdvice
public class GlobalExceptionHandler {
```



```
@ExceptionHandler(StudentNotFoundException.class)
public ResponseEntity<Object> handleStudentNotFoundException(StudentNotFoundException ex) {
    Map<String, Object> body = new HashMap<>();
    body.put("timestamp", LocalDateTime.now());
    body.put("status", HttpStatus.NOT_FOUND.value());
    body.put("error", "Not Found");
    body.put("message", ex.getMessage());

    return new ResponseEntity<>(body, HttpStatus.NOT_FOUND);
}
}
```

AccountController.java

```
package com.shriyash.mca.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import com.shriyash.mca.model.Account;
import com.shriyash.mca.service.AccountService;

@RestController
@RequestMapping("/api/accounts")
public class AccountController {

    @Autowired
    private AccountService accountService;

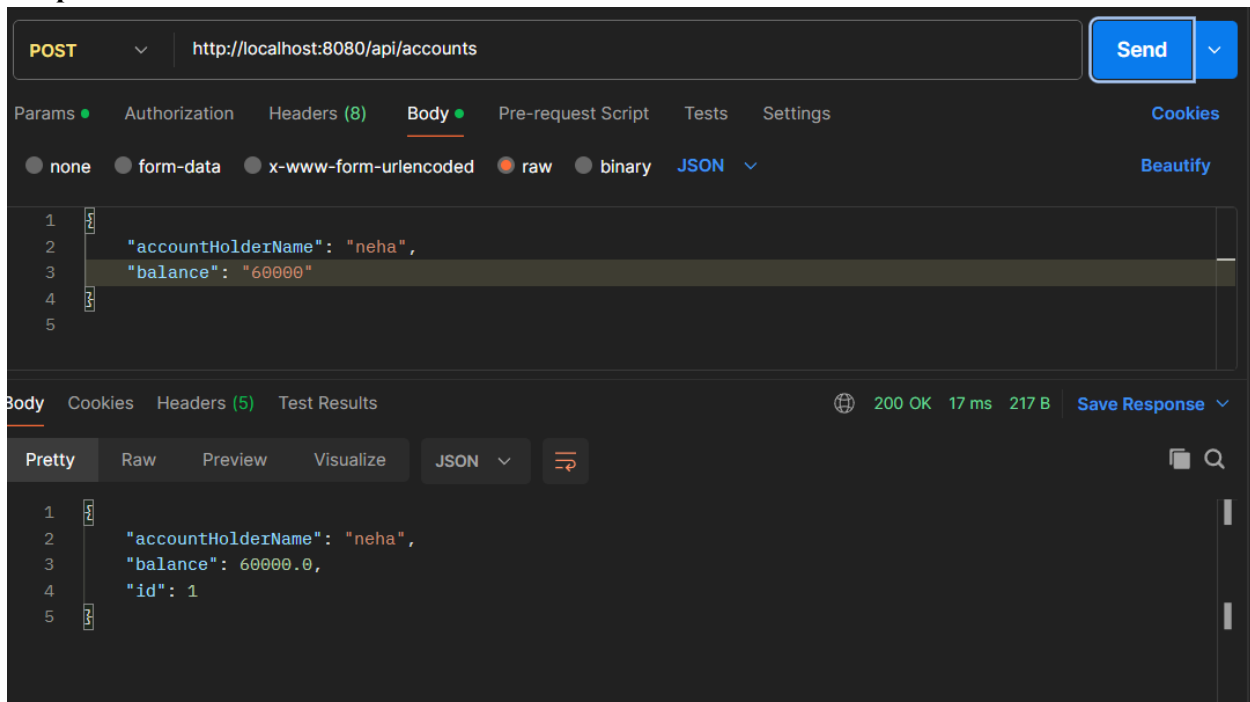
    @PostMapping
    public ResponseEntity<Account> createAccount(@RequestBody Account account) {
        return ResponseEntity.ok(accountService.createAccount(account));
    }

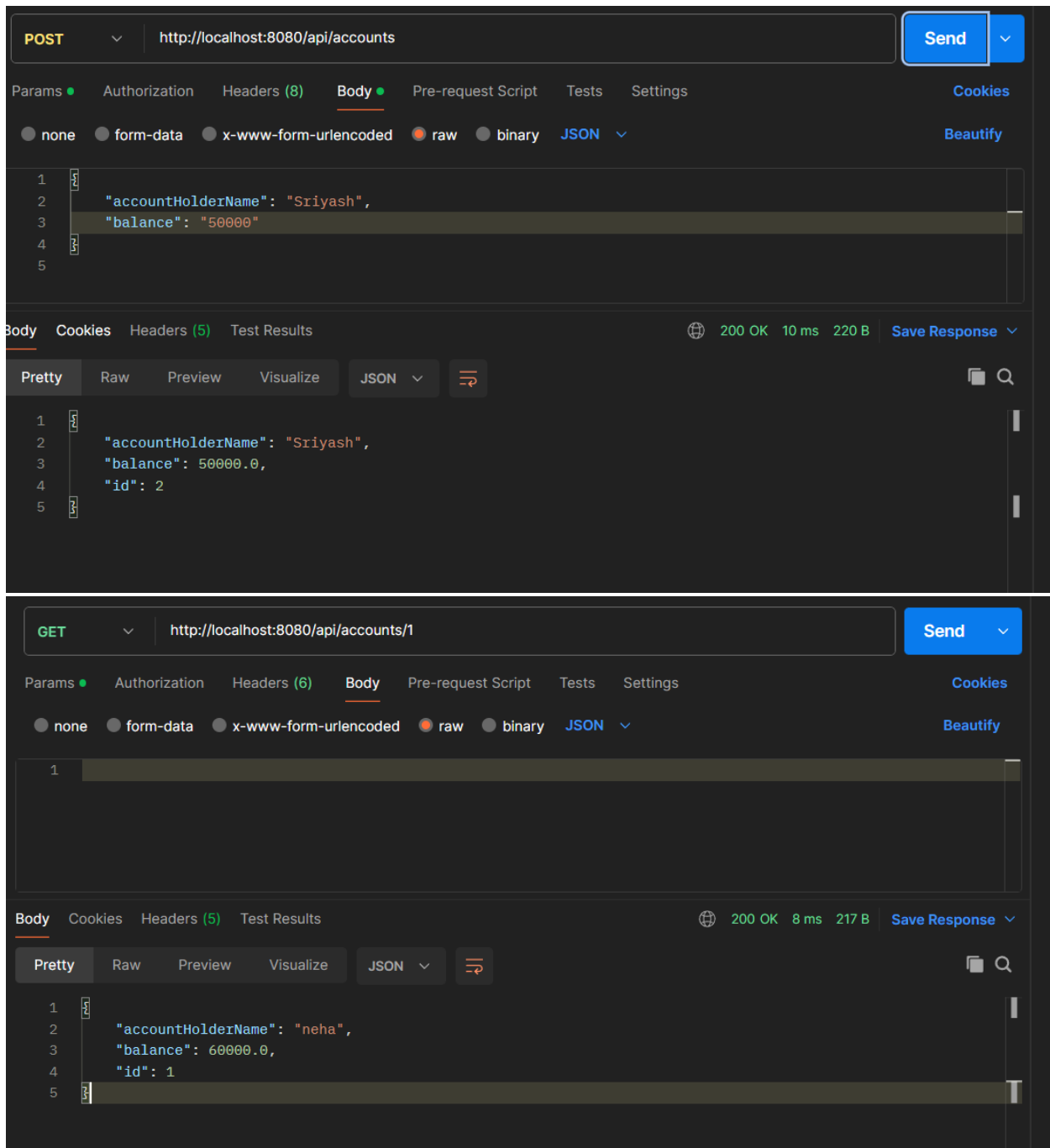
    @GetMapping("/{id}")
    public ResponseEntity<Account> getAccount(@PathVariable Long id) {
        return ResponseEntity.ok(accountService.getAccount(id));
    }
}
```

```
@PostMapping("/transfer")
public ResponseEntity<String> transferMoney(
    @RequestParam Long fromAccountId,
    @RequestParam Long toAccountId,
    @RequestParam Double amount,
    @RequestParam(defaultValue = "false") boolean simulateError) {

    accountService.transferMoney(fromAccountId, toAccountId, amount, simulateError);
    return ResponseEntity.ok("Transfer successful");
}
```

Output:





Conclusion

The implementation of Global Exception Handling and Transaction Management in Spring Boot successfully demonstrated advanced concepts used in enterprise-level REST API development. The Student REST API project showed how custom exceptions such as `StudentNotFoundException` can be created and handled globally using `@ControllerAdvice` and `@ExceptionHandler`.